

Etape 1:Chargement des données

importation de les bibliotheque nécessaire

```
In [224... import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt
import numpy as np
```

Chargement de dataset

```
In [225... """ l'explication de les colonnes:
1)
Max_BPM:Le maximum du rythme cardiaque atteint pendant la séance.
Mesurer l'intensité maximale de l'effort.
Plus le Max_BPM est élevé → plus la séance était intense → plus de calories brûlées.
=====
2)
Avg_BPM:Fréquence cardiaque moyenne pendant l'entraînement
Indique l'effort général durant la séance.
Un Avg_BPM élevé signifie une activité physique plus intense → brûle plus de calories.
=====
3)
Resting_BPM:Fréquence cardiaque au repos
Le nombre de battements par minute avant l'exercice.
Un sportif a souvent un Resting_BPM plus bas (50-60 bpm)
Une personne moins sportive → Resting_BPM plus haut (70-90 bpm)
=====
4)
Session_Duration(hours):Durée de la séance d'entraînement
5)
Fat_Percentage :Pourcentage de graisse corporelle
```

Indiquer la composition corporelle de la personne.

Homme moyen : 15-20%

Femme moyenne : 20-30%

Sportifs : 10-12% (H), 15-20%

Plus le taux de graisse est élevé → métabolisme plus lent → moins de calories brûlées, en général.

=====

6)

Water_Intake (liters):Quantité d'eau bue pendant la journée

=====

7)

Workout_Frequency (days/week):Nombre de séances par semaine

8)

Experience_Level:Niveau d'expérience (Beginner, Intermediate, Advanced)

9)

BMI:Indice de masse corporelle

Body Mass Index = Poids / (Taille²)

Mesurer si la personne est :

< 18.5 → maigre

18.5-24.9 → normal

25-29.9 → surpoids

30 → obésité

Influence la dépense énergétique :

Un BMI élevé → corps dépense plus d'énergie pour bouger → peut brûler plus de calories

Mais si trop élevé → condition physique faible → effort moins intense → brûle moins

""

```
Out[225... " l'explication de les colonnes:\n1)\nMax_BPM:Le maximum du rythme cardiaque atteint pendant la séance.\nMesurer l'intensité
maximale de l'effort.\nPlus le Max_BPM est élevé → plus la séance était intense → plus de calories brûlées.\n=====
=====
\n2)\nAvg_BPM:Fréquence cardiaque moyenne pendant l'entr
aînement\nIndique l'effort général durant la séance.\nUn Avg_BPM élevé signifie une activité physique plus intense → brûle pl
us de calories.\n=====
\n3)\nResting_BPM:Fré
quence cardiaque au repos\nLe nombre de battements par minute avant l'exercice.\nUn sportif a souvent un Resting_BPM plus bas
(50-60 bpm)\nUne personne moins sportive → Resting_BPM plus haut (70-90 bpm)\n=====
=====
\n4)\nSession_Duration(hours):Durée de la séance d'entraînement\n5)\nFat_Percenta
ge\t:Pourcentage de graisse corporelle\nIndiquer la composition corporelle de la personne.\nHomme moyen : 15-20%\nFemme moyen
ne : 20-30%\nSportifs : 10-12% (H), 15-20% \nPlus le taux de graisse est élevé → métabolisme plus lent → moins de calories br
ûlées, en général.\n=====
\n6)\nW
ater_Intake (liters):Quantité d'eau bue pendant la journée\n=====
=====
\n7)\nWorkout_Frequency (days/week):Nombre de séances par semaine\n8)\nExperience_Level:Nive
au d'expérience (Beginner, Intermediate, Advanced)\n9)\nBMI:Indice de masse corporelle\nBody Mass Index = Poids / (Taille²)\n
Mesurer si la personne est :\n< 18.5 → maigre\n18.5-24.9 → normal\n25-29.9 → surpoids\n30 → obésité\nInfluence la dépense éne
rgétique :\nUn BMI élevé → corps dépense plus d'énergie pour bouger → peut brûler plus de calories\nMais si trop élevé → cond
ition physique faible → effort moins intense → brûle moins\n"
```

```
In [226... try:
    df = pd.read_csv('gym_members_exercise_tracking.csv')
    print("Dataset chargé avec succès !")
except FileNotFoundError:
    print("Erreur : Le fichier 'gym_members_exercise_tracking.csv' n'a pas été trouvé.")
    print("Veuillez vous assurer que le fichier est dans le même répertoire que ce script ou fournir le chemin complet.")
    exit() # Quitte le script si le fichier n'est pas trouvé
df
```

Dataset chargé avec succès !

Out[226...

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Workout_Type	Fat_Percentage
0	56	Male	88.3	1.71	180	157	60	1.69	1313.0	Yoga	12.6
1	46	Female	74.9	1.53	179	151	66	1.30	883.0	HIIT	33.9
2	32	Female	68.1	1.66	167	122	54	1.11	677.0	Cardio	33.4
3	25	Male	53.2	1.70	190	164	56	0.59	532.0	Strength	28.8
4	38	Male	46.1	1.79	188	158	68	0.64	556.0	Strength	29.2
...	...	...	...	...	...	...	...	...	...	...	...
968	24	Male	87.1	1.74	187	158	67	1.57	1364.0	Strength	10.0
969	25	Male	66.6	1.61	184	166	56	1.38	1260.0	Strength	25.0
970	59	Female	60.4	1.76	194	120	53	1.72	929.0	Cardio	18.8
971	32	Male	126.4	1.83	198	146	62	1.10	883.0	HIIT	28.2
972	46	Male	88.7	1.63	166	146	66	0.75	542.0	Strength	28.8

973 rows × 15 columns



les information sur cette dataset:

In [227...

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 973 entries, 0 to 972
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   973 non-null    int64
1   Gender                               973 non-null    object
2   Weight (kg)                          973 non-null    float64
3   Height (m)                           973 non-null    float64
4   Max_BPM                              973 non-null    int64
5   Avg_BPM                              973 non-null    int64
6   Resting_BPM                          973 non-null    int64
7   Session_Duration (hours)             973 non-null    float64
8   Calories_Burned                      973 non-null    float64
9   Workout_Type                         973 non-null    object
10  Fat_Percentage                       973 non-null    float64
11  Water_Intake (liters)                 973 non-null    float64
12  Workout_Frequency (days/week)        973 non-null    int64
13  Experience_Level                      973 non-null    int64
14  BMI                                   973 non-null    float64
dtypes: float64(7), int64(6), object(2)
memory usage: 114.2+ KB

```

In [228... `df.describe()`

Out[228...

	Age	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Fat_Percentage	Water
count	973.000000	973.000000	973.00000	973.000000	973.000000	973.000000	973.000000	973.000000	973.000000	973
mean	38.683453	73.854676	1.72258	179.883864	143.766701	62.223022	1.256423	905.422405	24.976773	2
std	12.180928	21.207500	0.12772	11.525686	14.345101	7.327060	0.343033	272.641516	6.259419	0
min	18.000000	40.000000	1.50000	160.000000	120.000000	50.000000	0.500000	303.000000	10.000000	1
25%	28.000000	58.100000	1.62000	170.000000	131.000000	56.000000	1.040000	720.000000	21.300000	2
50%	40.000000	70.000000	1.71000	180.000000	143.000000	62.000000	1.260000	893.000000	26.200000	2
75%	49.000000	86.000000	1.80000	190.000000	156.000000	68.000000	1.460000	1076.000000	29.300000	3
max	59.000000	129.900000	2.00000	199.000000	169.000000	74.000000	2.000000	1783.000000	35.000000	3



Etape 2:Cleaning

les Valeurs Manquantes

In [229...

```
df.isnull().sum()  
# cette dataset ne contient pas des valeurs manquantes
```

```
Out[229... Age          0
           Gender       0
           Weight (kg)   0
           Height (m)    0
           Max_BPM       0
           Avg_BPM       0
           Resting_BPM   0
           Session_Duration (hours) 0
           Calories_Burned 0
           Workout_Type  0
           Fat_Percentage 0
           Water_Intake (liters) 0
           Workout_Frequency (days/week) 0
           Experience_Level 0
           BMI           0
           dtype: int64
```

Les doublons

```
In [230... print(f"Nombre de lignes dupliques : {df.duplicated().sum()}")
```

Nombre de lignes dupliques : 0

Outliers

```
In [231... # Function traiter s'il existe des valeurs outliers
def verifier_outlier_IQR_Traiter(name_col):
    Q1 = df[name_col].quantile(0.25)
    Q3 = df[name_col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[name_col] < lower) | (df[name_col] > upper)]
    print(f"Nombre d'outliers dans la '{name_col}' est : {outliers.shape[0]}")
    if(outliers.shape[0]!=0):
        print(outliers[[name_col]].head(5))
        df[name_col].plot.box()
        plt.show()
    print("=====")
```

```
# Traitement des Outliers
df[name_col]=df[name_col].clip(lower=lower,upper=upper)
```

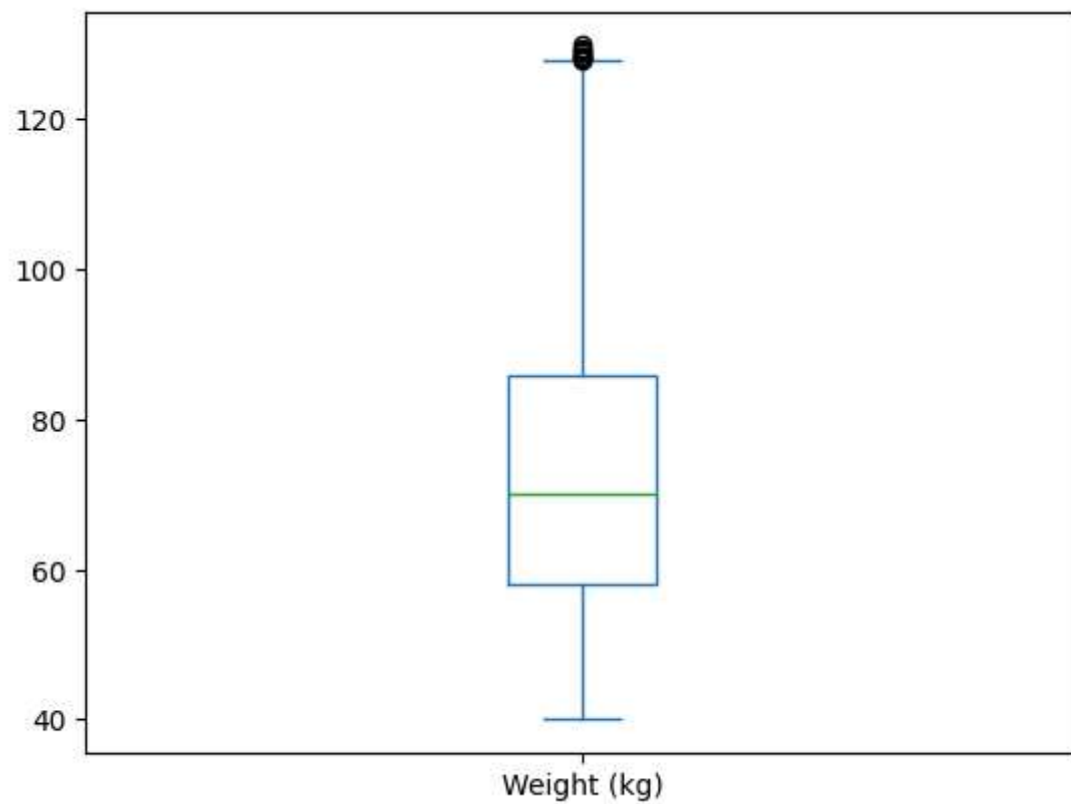
```
In [232... columns_num=df.drop(columns=['Gender', 'Workout_Type'])
for col in columns_num:
    verifier_outlier_IQR_Traiter(col)
    print("=====")
```

Nombre d'outliers dans la 'Age'est : 0

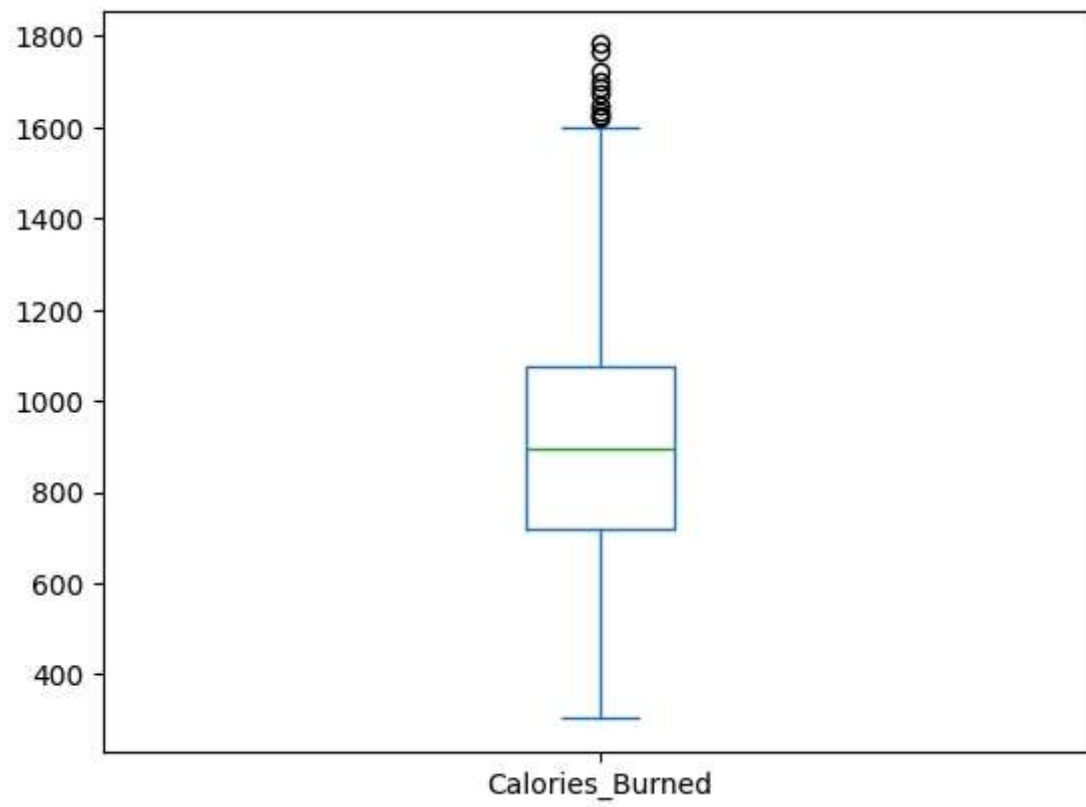
=====

Nombre d'outliers dans la 'Weight (kg)'est : 9

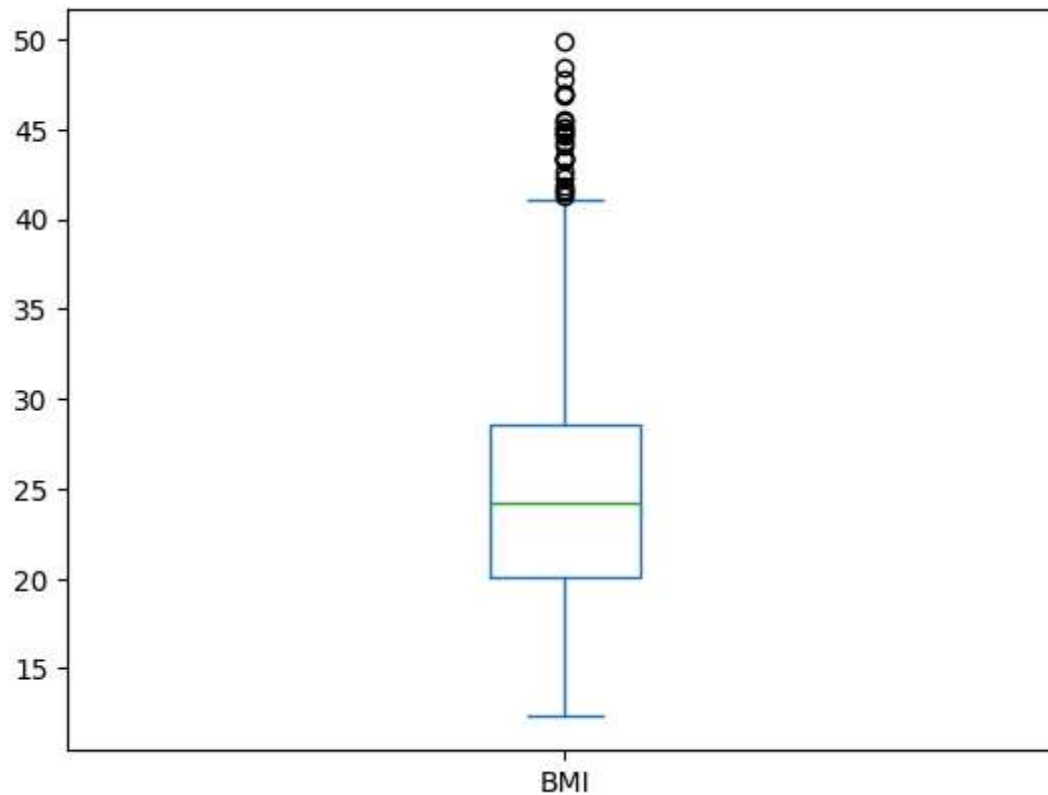
	Weight (kg)
96	129.0
122	129.5
180	128.2
283	128.4
291	128.4



```
=====
=====
Nombre d'outliers dans la 'Height (m)'est : 0
=====
Nombre d'outliers dans la 'Max_BPM'est : 0
=====
Nombre d'outliers dans la 'Avg_BPM'est : 0
=====
Nombre d'outliers dans la 'Resting_BPM'est : 0
=====
Nombre d'outliers dans la 'Session_Duration (hours)'est : 0
=====
Nombre d'outliers dans la 'Calories_Burned'est : 10
    Calories_Burned
90      1688.0
99      1625.0
124     1701.0
475     1622.0
511     1725.0
```



```
=====
=====
Nombre d'outliers dans la 'Fat_Percentage'est : 0
=====
Nombre d'outliers dans la 'Water_Intake (liters)'est : 0
=====
Nombre d'outliers dans la 'Workout_Frequency (days/week)'est : 0
=====
Nombre d'outliers dans la 'Experience_Level'est : 0
=====
Nombre d'outliers dans la 'BMI'est : 25
      BMI
10    43.31
12    43.40
35    42.63
55    44.84
133   45.43
```



=====

Etape 3:Transformation

```
In [233... # La colonne 'Workout_Type'  
df['Workout_Type'].value_counts()
```

```
Out[233... Workout_Type  
Strength    258  
Cardio      255  
Yoga        239  
HIIT        221  
Name: count, dtype: int64
```

```
In [234... # Appliquer LabelEncoder a Gender:
le=LabelEncoder()
df['Gender']=le.fit_transform(df['Gender'])
# Appliquer OneHotEncoder a 'Workout_Type'
ohe=OneHotEncoder(sparse_output=False)
Workout_Type_encoded=ohe.fit_transform(df[['Workout_Type']])
Workout_Type_cols = ohe.get_feature_names_out(['Workout_Type'])
df_Workout_Type_ohe = pd.DataFrame(Workout_Type_encoded, columns=Workout_Type_cols)
# Fusionner avec df
df = pd.concat([df.drop(columns=['Workout_Type']), df_Workout_Type_ohe], axis=1)
df.head()
```

Out[234...

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Fat_Percentage	Water_Intake (liters)	V
0	56	1	88.3	1.71	180	157	60	1.69	1313.0	12.6	3.5	
1	46	0	74.9	1.53	179	151	66	1.30	883.0	33.9	2.1	
2	32	0	68.1	1.66	167	122	54	1.11	677.0	33.4	2.3	
3	25	1	53.2	1.70	190	164	56	0.59	532.0	28.8	2.1	
4	38	1	46.1	1.79	188	158	68	0.64	556.0	29.2	2.8	

Etape 4: Split train/test

```
In [235... # Separation de donnee:
X=df.drop(columns='Calories_Burned')
y=df['Calories_Burned'].values
# StandardScaler
scaler_X = StandardScaler()
X_scaled = scaler_X.fit_transform(X)

# Division:
X_train,X_test,y_train,y_test=train_test_split(X_scaled,y,test_size=0.3,random_state=42)
```

Etape 5:Entrainement du modele Decision Tree

```
In [236... # on creer un model de arbre de decision avec Le constuctor "DecisionTreeRegressor()"
# min_samples_split: Nombre minimum d'échantillons requis pour diviser un nœud interne
# min_samples_leaf: Nombre minimum d'échantillons requis pour être un nœud feuille
# random_state: Pour la reproductibilité
model=DecisionTreeRegressor(
    criterion='squared_error',
    max_depth=5,
    min_samples_leaf=5,
    random_state=42
)
model.fit(X_train,y_train)

#Prediction:
y_pred=model.predict(X_test)
```

Etape 6: Evaluation du modele

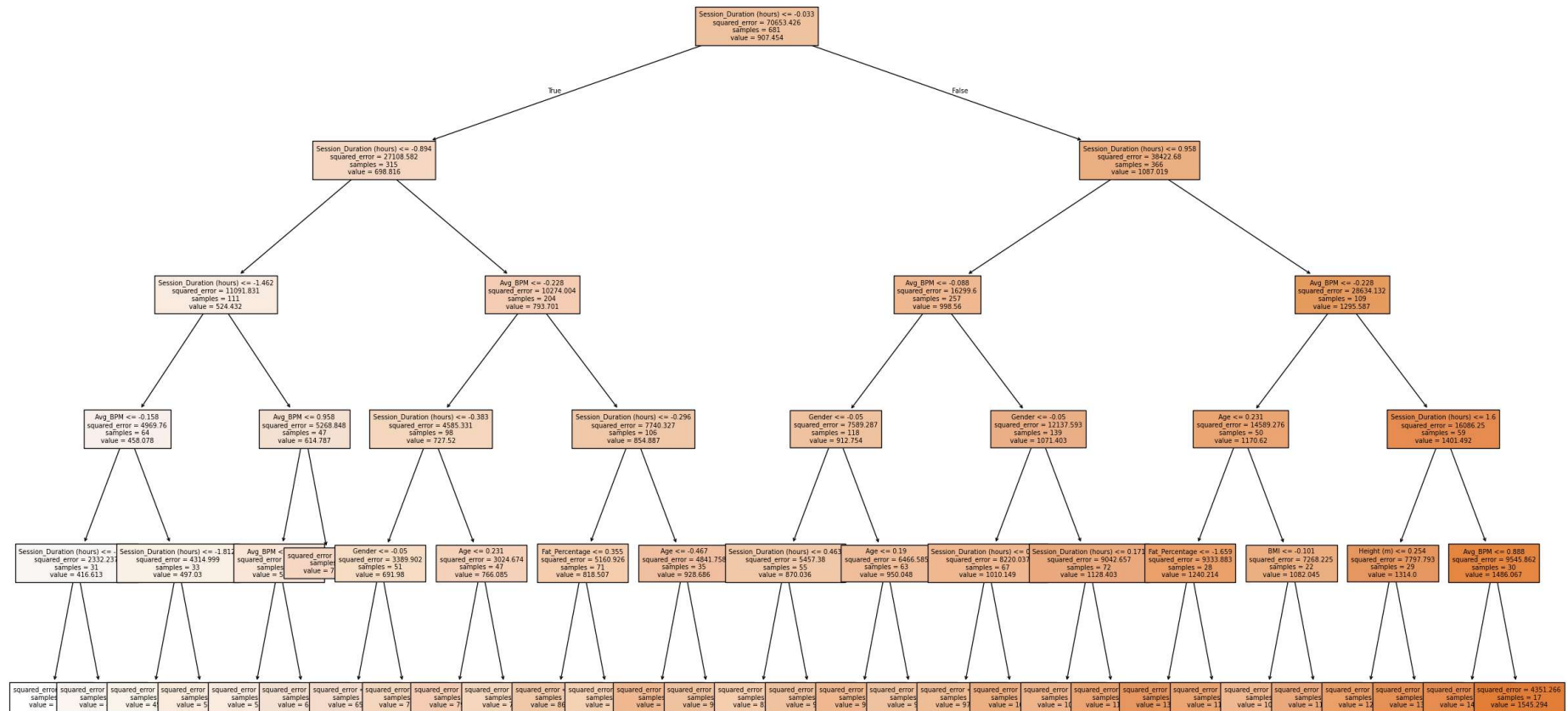
```
In [237... # Calculer les métriques d'évaluation
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"\n--- Évaluation du Modèle ---")
print(f"Mean Squared Error (MSE) : {mse:.2f}")
print(f"Root Mean Squared Error (RMSE) : {rmse:.2f}")
print(f"R-squared (R2) : {r2:.2f}")
```

```
--- Évaluation du Modèle ---
Mean Squared Error (MSE) : 7052.43
Root Mean Squared Error (RMSE) : 83.98
R-squared (R2) : 0.91
```

```
In [238... plt.figure(figsize=(30,16))
plot_tree(model, feature_names=X.columns,class_names=['Calories_Burned'], filled=True,fontsize=7)
```

```
plt.show()
```



```
In [239... import joblib
joblib.dump(model, 'decTreeReg_model.pkl')

print("Modele sauvegarde sous le nom 'decTreeReg_model.pkl')"
```

Modele sauvegarde sous le nom 'decTreeReg_model.pkl'